

КАФКА НА ПАЛЬЦАХ

Живые стенды внутри PDF-документа

В PDF нет ни HTML, ни CSS — только поля форм и собственный JavaScript. Стенды на следующих страницах собраны из них: клетки логга это кнопки, движение — таймер, подписи — текстовые поля. Нажимай кнопки, они настоящие.

append-only



ПРОВЕРКА ПРОСМОТРЩИКА

Скрипты не запущены — здесь нужен Chrome, Edge или Acrobat Reader

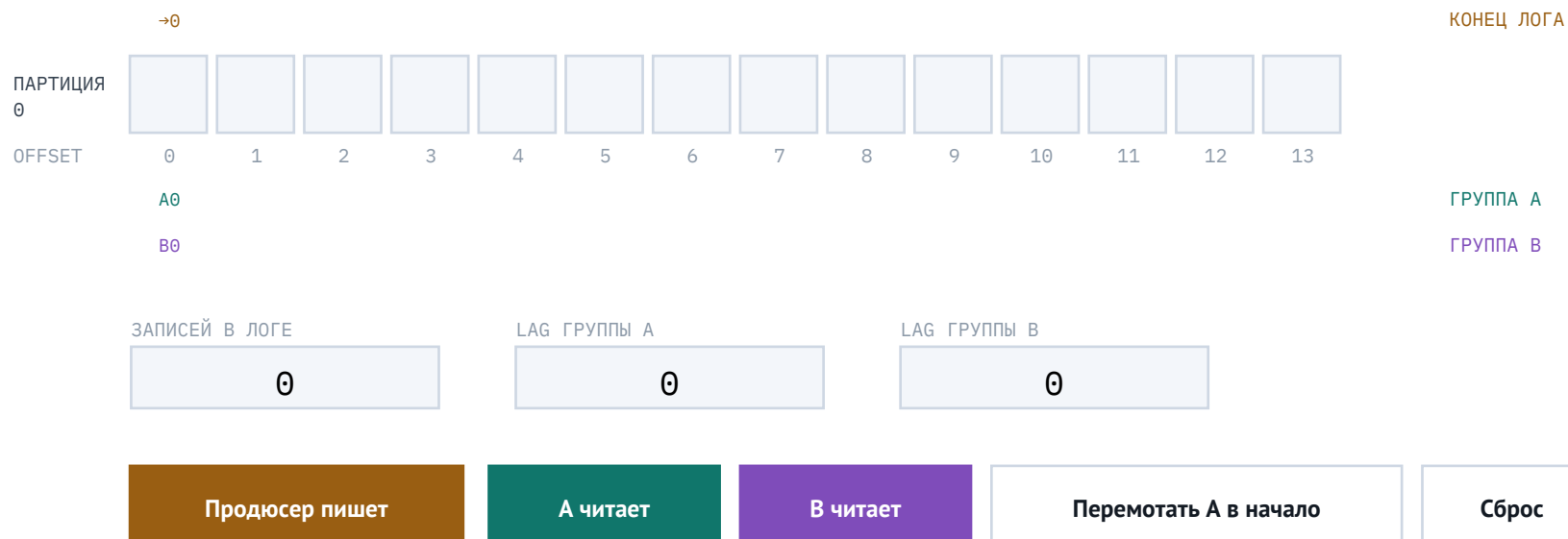
Если строка выше сменилась — всё работает. Если осталась такой, как есть, значит просмотрщик не умеет исполнять PDF-скрипты: так ведут себя Просмотр на macOS, встроенные читалки телефонов и предпросмотр внутри соцсетей. Скачай файл и открой в браузере на компьютере.

Листай дальше →

СТЕНД 01

Лог, а не очередь

Чтение ничего не удаляет. Продюсер дописывает в конец, а каждый читатель держит собственную закладку и двигает только её. Отсюда и перемотка, и независимые потребители.



Демонстрация идёт сама — нажми любую кнопку, чтобы взять управление

Обрати внимание: когда А читает, клетка остаётся на месте — двигается только закладка. Кнопка перемотки возвращает А в начало, и вся история читается заново. В очереди такой кнопки не существует: там прочитанное удалено.

Ничего не двигается? Открой этот файл в Chrome, Edge или Acrobat Reader — в них PDF умеет исполнять скрипты.

Ключ решает партицию

Продюсер не выбирает партицию руками — за него это делает ключ: номер = hash(ключ) % число партиций. Что из этого следует для порядка записей и что случается, когда один ключ перетягивает всё на себя?

OFFSET

01234567

ПАРТИЦИЯ 0

ПАРТИЦИЯ 1

ПАРТИЦИЯ 2

01234567

0

0

0

ЗАПИСЕЙ

0

0

0

КЛЮЧ → ПАРТИЦИЯ

user-1 → партиция 2

crc32 2116437524 % 3 = 2

user-7 → партиция 0

crc32 2537939745 % 3 = 0

user-42 → партиция 1

crc32 2097592435 % 3 = 1

? — без ключа, по кругу

КАК ВЫБРАНА ПАРТИЦИЯ

hash(ключ) % 3 = номер партиции

user-1

user-7

user-42

без ключа

Перекоc: 6 раз user-42

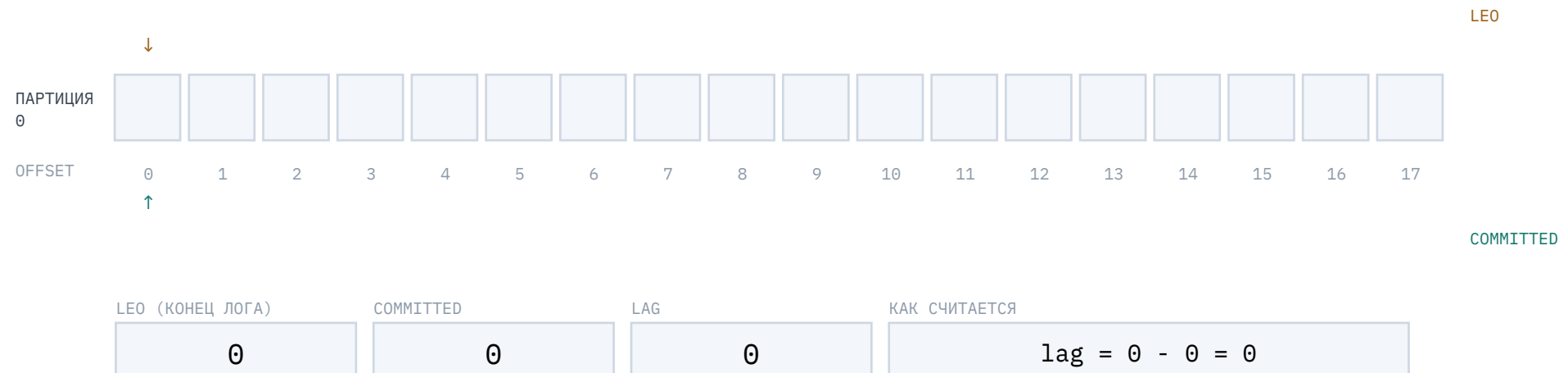
Сброс

Демонстрация идёт сама — нажми любую кнопку, чтобы взять управление

Хеш берётся от байтов ключа: пока число партиций не меняется, ключ всегда приводит в одну и ту же партицию. Добавили партиций — делитель другой, тот же ключ уезжает в другую ленту, а записанное раньше остаётся на старом месте. Разные ключи могут съехаться в одну партицию: хеш обещает стабильность адреса, а не разные адреса. Запись без ключа продюсер раскладывает сам — здесь по кругу, в нынешней Kafka «липкими» пачками. Хеш у Kafka — murmur2, здесь для наглядности crc32: арифметика та же.

Lag: насколько устарели твои данные

Сколько сообщений консьюмер ещё не прочитал прямо сейчас? Lag — это разница между концом лога и сохранённой позицией группы, и именно он отвечает на вопрос «насколько свежи мои данные».



Продюсер быстрее

Консьюмер быстрее

Уронить консьюмера

Поднять консьюмера

Сброс

Симуляция уже идёт: продюсер пишет по одной записи за такт, консьюмер читает по одной. Нажми любую кнопку, чтобы сбить равновесие.

Committed offset хранится не у консьюмера, а в самой Kafka — в служебном топике `__consumer_offsets`. Поэтому упавший консьюмер поднимается и продолжает с сохранённой позиции, а не с нуля. И ещё тонкость: committed — это номер СЛЕДУЮЩЕГО сообщения для чтения, а не последнего обработанного. Лента здесь — окно на 18 позиций бесконечного лога: когда конец лога упирается в правый край, окно едет вправо и номера под клетками меняются вместе с ним — offset при этом только растёт, как и в Kafka. А если отставание переросло ленту, конец лога уходит за правый край: рисовать его негде, но числа считают дальше — потолка у lag нет, пока записи не удалит retention.

Ничего не двигается? Открой этот файл в Chrome, Edge или Acrobat Reader — в них PDF умеет исполнять скрипты.

acks=all обманчив без min.insync.replicas

Продюсер ждёт подтверждения «всех реплик» — но всех из ISR, а список ISR умеет схлопываться до одного лидера. Сколько копий у записи, за которую уже сказали «ок»?

ТОПИК ORDERS, ПАРТИЦИЯ 0, REPLICATION.FACTOR = 3

БРОКЕР 1
ЛИДЕР

лидер, жив

БРОКЕР 2
РЕПЛИКА

в ISR

БРОКЕР 3
РЕПЛИКА

в ISR

ACKS — ЧЕГО ЖДЁТ ПРОДЮСЕР

0

1

all

all

MIN.INSYNC.REPLICAS — МИНИМУМ РЕПЛИК В ISR

1

2

3

1

ISR — РЕПЛИКИ, КОТОРЫЕ УСПЕВАЮТ ЗА ЛИДЕРОМ

ISR = [1, 2, 3]

вердикт появится, когда убьёшь лидера

Записать

Убить лидера

Реплики отстали

Реплики догнали

Надёжная тройка

Сброс

Демонстрация идёт сама — нажми любую кнопку, чтобы взять управление

acks=all не обещает трёх копий — он обещает дождаться тех, кто сейчас в ISR. Схлопнулся ISR до одного лидера — и «все» означает одного, а продюсер всё равно получает ок. Явный отказ лучше тихой потери: replication.factor = 3, acks = all, min.insync.replicas = 2.

ЭТО БЫЛА КОРОТКАЯ ВЕРСИЯ

Полный разбор – пятнадцать глав в браузере

Партиции и ключи, consumer groups и ребалансы, retention и compaction, лидер и ISR, acks и тихая потеря записи, диагностика lag по партициям, песочница и шпаргалка для собеседования. Всё интерактивное, открывается в любом браузере и работает на телефоне.

warhammer2000.github.io/kafka-na-palcah

А этот PDF — побочный опыт: интерактив внутри документа собран из полей формы и документного JavaScript, потому что ни HTML, ни CSS в PDF не существует. Исходники сборщика лежат там же, в репозитории.